

NewsBrief: Redesign & Refactoring

Blueprint

Zero-Ambiguity Engineering Roadmap for Google Antigravity IDE.

Transforming from a consumer news aggregator into an Autonomous Market & Competitor Intelligence SaaS for Tech Leaders, Founders, and Product Teams.

System Architect: Antigravity AI

Target Repository: /home/rohan/Desktop/StartupX

Refactoring Checkpoints: CP-16 through CP-25 (Deterministic Execution)

Document Version: 3.0 (Comprehensive Implementation-Ready Edition)

Date: September 2026

1. Architectural Evolution & Zero-Assumption Principles

This specification provides an exact, deterministic instruction manual for Google Antigravity IDE. No design decisions are left unspecified. Every checkpoint defines **Target Files**, **Dependencies**, **Data Models**, **Exact Algorithms/Signatures**, and **Verification Commands**.

1.1 The Fundamental Structural Problem

Currently, `backend/run_pipeline.py` executes a linear loop per registered user:

For each User:

For each Search Query (5-8 queries):

Serper.dev News Search (3 links)

Jina Reader Scrape (3 pages)

sentence-transformers Deduplication

Groq LLM Thematic Clustering

Groq LLM Card Generation

Failure Modes:

- **Quadratic Scaling & Rate Limits:** 1,000 users with overlapping interests trigger 6,000 Serper searches, 18,000 Jina scrapes, and 6,000 Groq cluster calls. Serper and Jina API keys will be throttled within minutes, and monthly costs will exceed \$1,500.
- **SEO Spam & PR Farm Poisoning:** Serper returns syndicated PR newswires and content-farm blog posts, degrading summary quality for technical founders.
- **Single Point of Failure Delivery:** Only delivering via Mobile Push means if an iOS or Android user denies notification permissions, engagement drops to zero.

1.2 The Two-Stage Decoupled World-State Architecture

We decouple the system into two distinct operational stages running on separate schedules:

STAGE 1: GLOBAL WORLD-STATE INGESTION & CLUSTERING (Daily at 03:00 UTC)

| Sources: Hacker News (Firebase), GitHub Trending, arXiv Preprints, Curated Tech RSS |

| Pipeline: Ingest All Raw Articles (Pool: ~500) -> Semantic Dedup -> Groq Clustering |

| Output: 50-80 Canonical NewsClusters with 384-dim Centroid Embeddings in PostgreSQL |

| Pre-generation: Base Cards & Top-10 In-depth Research Deep Dives generated ONCE. |

STAGE 2: PERSONALIZED USER DELIVERY ENGINE (Daily at 06:30 Local User Time)

| 1. Vector Match: User Preference Embedding <=> Cluster Embeddings (via pgvector) |

| 2. Select: Top 5-7 highest similarity clusters for the individual user |

| 3. Synthesize: User-tailored Super Summary in user's configured tone bucket (1.2s LLM) |

| 4. Dispatch: Multi-channel delivery to Email Digest, Mobile Push, Audio TTS, Slack Bot |

2. Complete Engineering Checkpoints (CP-16 to CP-25)

Checkpoint 16: Multi-Source High-Signal Ingestion Engine Backend Pipeline

OBJECTIVE & SCOPE

Build modular, robust scrapers for Hacker News, GitHub Trending, arXiv, and Substack RSS to feed the daily

raw article pool, eliminating exclusive dependency on Serper.

FILES TO CREATE / MODIFY

- [NEW] backend/ingestion_sources/__init__.py
- [NEW] backend/ingestion_sources/hn.py — Hacker News Firebase REST API reader.
- [NEW] backend/ingestion_sources/github.py — GitHub Trending repos & releases crawler.
- [NEW] backend/ingestion_sources/arxiv.py — arXiv API client & XML abstract extractor.
- [NEW] backend/ingestion_sources/rss.py — Substack / Tech Blog RSS parser via feedparser.
- [MODIFY] backend/requirements.txt — Add feedparser==6.0.11, httpx==0.27.0.
- [MODIFY] backend/schemas.py — Add RawArticle Pydantic model.

- [NEW] backend/tests/test_ingestion_sources.py — Automated unit tests for all 4 sources.

DATA CONTRACTS & SCHEMAS (BACKEND/SCHEMAS.PY)

```
class RawArticle(BaseModel):
    """
    Raw article data as received from ingestion sources.
    """

    id: str = Field(..., description="Unique hash or source-specific ID")

    title: str = Field(..., description="Article title")

    url: str = Field(..., description="Source URL")

    source_name: str = Field(..., description="Source name: 'Hacker News', 'GitHub', 'arXiv', 'Substack: Stratechery'")

    content: str = Field(..., description="Clean text or markdown body (min 150 chars)")

    published_at: Optional[datetime] = None

    tags: List[str] = Field(..., description="List of tags")

    author: Optional[str] = None

    score: Optional[int] = 0 # Upvotes / Stars
```

EXACT IMPLEMENTATION INSTRUCTIONS

-
- **HN Module (hn.py):** Call <https://hacker-news.firebaseio.com/v0/topstories.json>. Fetch item details for the top
-

30 IDs from <https://hacker-news.firebaseio.com/v0/item/{id}.json>. Filter for stories with score ≥ 30 .

Extract URL and title; if story is a text post (Ask HN / Show HN), use item.text as content. If external

link, call Jina Reader with 5s timeout.

-
- **GitHub Module (github.py):** Call GitHub Search API <https://api.github.com/search/repositories?>
-

`q=created:>{7_days_ago}+stars:>100&sort=stars&order=desc`. Map top 15 repositories to RawArticle with

content set to repo description, primary language, star count, and README.md first 800 characters.

-
- **arXiv Module (arxiv.py):** Call arXiv API <http://export.arxiv.org/api/query?>
-

`search_query=cat:cs.AI+OR+cat:cs.LG+OR+cat:cs.CL&start=0&max_results=25&sortBy=submittedDate&sortOrder=des`

cending. Parse XML using `xml.etree.ElementTree`. Extract title, abstract summary, published date, PDF

link, and authors.

-
- **RSS Module (rss.py):** Maintain a static list of 20 verified high-signal RSS URLs (e.g. Stratechery,
-

SemiAnalysis, Latent Space, Pragmatic Engineer, Simon Willison). Use `feedparser.parse(url)` with an

HTTP ETag / If-Modified-Since cache header. Extract articles published in the last 36 hours.

VERIFICATION COMMAND

```
pytest backend/tests/test_ingestion_sources.py -v
```

Expected: 4 tests passed, >60 unique RawArticle items returned across all 4 modules.

Checkpoint 17: Database Schema Migration & pgvector Integration Database

OBJECTIVE & SCOPE

Enable the PostgreSQL vector extension on the database, add vector columns for 384-dimensional

sentence embeddings, and create relational schemas for B2B Teams and Multi-Channel delivery

preferences.

FILES TO CREATE / MODIFY

- [MODIFY] backend/requirements.txt — Add pgvector==0.3.6.
- [MODIFY] backend/db.py — Ensure engine registers pgvector and extension is created on startup.
- [MODIFY] backend/models.py — Update NewsCluster, UserPreference, SuperSummary; add Team, TeamMembership, SlackInstallation, UserEmailPreference.
- [NEW] backend/alembic/versions/0002_add_pgvector_and_b2b_models.py — Alembic migration script.

- [NEW] backend/tests/test_database_vector.py — Test script verifying cosine vector search.

EXACT ORM MODEL SPECIFICATIONS (BACKEND/MODELS.PY)

```
from pgvector.sqlalchemy import Vector
```

```
# Alterations to existing tables
```

```
class NewsCluster(Base):
```

```
# Existing columns ...
```

```
embedding = Column(Vector(384), nullable=True) # Normalized centroid embedding
```

```
category = Column(String, default="General Tech", nullable=False)
```

```
class UserPreference(Base):
```

```
# Existing columns ...
```

```
preference_embedding = Column(Vector(384), nullable=True) # Normalized user profile vector
```

```
class SuperSummary(Base):
```

```
# Existing columns ...
```

```
audio_url = Column(String, nullable=True)
```

```
audio_duration_seconds = Column(Integer, nullable=True)
```

```
# New B2B & Delivery Tables
```

```
class Team(Base):
```

```
    __tablename__ = "teams"
```

```
    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```
    name = Column(String, nullable=False)
```

```
    created_at = Column(DateTime(timezone=True), default=lambda: datetime.now(timezone.utc))
```

```
    subscription_status = Column(String, default="active") # 'active', 'past_due', 'canceled'
```

```
    max_seats = Column(Integer, default=15)
```

```
    billing_email = Column(String, nullable=True)
```

```
class TeamMembership(Base):
```

```
    __tablename__ = "team_memberships"
```

```
    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```
team_id = Column(UUID(as_uuid=True), ForeignKey("teams.id", ondelete="CASCADE"), nullable=False)
```

```
user_id = Column(UUID(as_uuid=True), ForeignKey("users.id", ondelete="CASCADE"), nullable=False)
```

```
role = Column(String, default="member") # 'admin', 'member'
```

```
class SlackInstallation(Base):
```

```
    __tablename__ = "slack_installations"
```

```
    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
```

```
    team_id = Column(UUID(as_uuid=True), ForeignKey("teams.id", ondelete="CASCADE"), nullable=True)
```

```
    slack_team_id = Column(String, unique=True, nullable=False)
```

```
    slack_team_name = Column(String, nullable=False)
```

```
    bot_token = Column(String, nullable=False)
```

```
    target_channel_id = Column(String, nullable=False)
```

```
    delivery_time_utc = Column(String, default="13:00") # 9:00 AM EST default
```

```
    is_active = Column(Boolean, default=True)
```

```
class UserEmailPreference(Base):
```

```
__tablename__ = "user_email_preferences"
```

```
user_id = Column(UUID(as_uuid=True), ForeignKey("users.id", ondelete="CASCADE"), primary_key=True)
```

```
email_digest_enabled = Column(Boolean, default=True, nullable=False)
```

```
delivery_hour_utc = Column(Integer, default=11, nullable=False) # 7:00 AM EST default
```

```
last_sent_at = Column(DateTime(timezone=True), nullable=True)
```

ALEMBIC MIGRATION & VERIFICATION INSTRUCTIONS

```
# In PostgreSQL terminal or migration script
```

```
CREATE EXTENSION IF NOT EXISTS vector;
```

```
CREATE INDEX IF NOT EXISTS ix_news_clusters_embedding ON news_clusters USING ivfflat (embedding vector_cosine_ops) WITH (lists =
```

```
100);
```

VERIFICATION COMMAND

```
python3 -m pytest backend/tests/test_database_vector.py -v
```

Expected: Connection connects, vectors inserted, and SELECT ... ORDER BY embedding <=> '['...']' LIMIT 5 succeeds.

Checkpoint 18: Stage 1 Global World-State Pipeline Backend Pipeline

OBJECTIVE & SCOPE

Build the standalone, user-independent Stage 1 pipeline script. Ingests all sources, deduplicates, clusters,

generates centroid embeddings, and pre-generates base Cards and Deep Dives once into the database.

FILES TO CREATE / MODIFY

- [NEW] backend/pipeline_stage1.py — Main orchestrator for Stage 1 batch.
- [MODIFY] backend/clustering.py — Add centroid embedding computation (mean pooling + normalization).
- [MODIFY] backend/generation.py — Add batch card generation and priority Deep Dive generation.
- [NEW] backend/tests/test_pipeline_stage1.py — End-to-end stage 1 test with mock sources.

STEP-BY-STEP EXECUTION FLOW

1. **Aggregation:** Concurrently call `fetch_hacker_news()`, `fetch_github_trending()`, `fetch_arxiv_papers()`,

`fetch_rss_feeds()`. Combine into `List[RawArticle]` (~400–600 items).

2. **Deduplication:** Embed `f"{art.title}. {art.content[:400]}"` via sentence-transformers (all-MiniLM-L6-v2). Compute

cosine distance matrix. Retain only articles with similarity < 0.82 to any higher-ranked article.

-
3. **Clustering:** Pass surviving article titles + snippets to Groq (qwen/qwen3.8-27b). Prompt outputs 40–70
-

clusters with canonical titles, matched tags, and article index mappings.

4. **Centroid Embedding Calculation:**

```
def compute_centroid(article_embeddings: np.ndarray) -> List[float]:
```

```
    centroid = np.mean(article_embeddings, axis=0)
```

```
    norm = np.linalg.norm(centroid)
```

```
    if norm > 0:
```

```
        centroid = centroid / norm
```

```
    return centroid.tolist()
```

5. **Pre-generation:**

- For each cluster, generate base Card for high_signal and technical_deep tone.
-

- Select the top 10 clusters (ranked by source count + score) and pre-generate the structured
-

4-section DeepDive markdown.

6. **Database Upsert:** Save all NewsCluster, Card, and DeepDive records linked to batch_date = today in a

single transaction.

VERIFICATION COMMAND

```
python3 -m pytest backend/tests/test_pipeline_stage1.py -v
```

Expected: Pipeline generates 40-70 clusters with non-null embeddings and base cards in test DB.

Checkpoint 19: Stage 2 User Matching & Personalization Engine Backend Pipeline

OBJECTIVE & SCOPE

Build the fast Stage 2 personalization engine. Matches each user's preference vector against the day's pre-

computed clusters using pgvector, generates a custom Super Summary, and creates the UserBriefing.

FILES TO CREATE / MODIFY

- [NEW] backend/pipeline_stage2.py — High-speed personalization worker.
- [MODIFY] backend/main.py — Add POST /admin/trigger-stage2 and optimize GET /briefing/today.
- [NEW] backend/tests/test_pipeline_stage2.py — Benchmark test for user matching speed (<1.5s/user).

MATCHING QUERY & PERSONALIZATION LOGIC

```
# In backend/pipeline_stage2.py

def match_clusters_for_user(db: Session, user_id: UUID, limit: int = 6) -> List[models.NewsCluster]:
    """
    Match news clusters for a user based on their preferences.
    """
    pref = db.query(models.UserPreference).filter(models.UserPreference.user_id == user_id).first()

    # If user has no embedding, compute and save it once
    if pref.preference_embedding is None:
        text = f'{pref.raw_paragraph} {" ".join(pref.thematic_tags or [])} {" ".join(pref.search_queries or [])}'
        pref.preference_embedding = get_embedder().encode(text).tolist()

    db.commit()

    today = datetime.now(timezone.utc).date()

    # High-speed pgvector cosine distance query
    matched_clusters = db.query(models.NewsCluster).filter(
        models.NewsCluster.batch_date == today
```

```
        ).order_by(
            models.NewsCluster.embedding_cosine_distance(pref.preference_embedding)
        ).limit(limit,all=True)

    return matched_clusters
```

SUPER SUMMARY SYNTHESIS PROMPT (GROQ QWEN)

Synthesize the 6 matched cluster titles into a punchy, 4–6 sentence narrative strictly adhering to the user's

tone_bucket (e.g. technical_deep OR executive_brief).

VERIFICATION COMMAND

```
python3 -m pytest backend/tests/test_pipeline_stage2.py -v
```

Expected: 10 mock users with distinct tags (e.g. 'Kubernetes' vs 'Biotech') match distinct clusters in <1.2s each.

Checkpoint 20: Daily Email Digest Service (Resend Integration) Multi-Channel

OBJECTIVE & SCOPE

Implement high-converting, responsive daily HTML email delivery via Resend API, establishing email as the

primary retention vehicle.

FILES TO CREATE / MODIFY

- [MODIFY] backend/requirements.txt — Add resend==2.6.0.
- [NEW] backend/services/__init__.py
- [NEW] backend/services/email_service.py — Resend email dispatcher.
- [NEW] backend/templates/email_digest.html — Responsive HTML email template with dark-slate styling.
- [MODIFY] backend/main.py — Add GET /users/me/email-preferences and PUT /users/me/email-preferences.
- [NEW] backend/tests/test_email_service.py — Unit tests with mocked Resend API.

EMAIL ARCHITECTURE & CONFIGURATION

- **Environment Variables:** RESEND_API_KEY=re_..., FROM_EMAIL=briefing@newsbrief.ai.
- **Template Layout:**

- Header: NewsBrief Logo, Date, User Name.

-
- Section 1: *Executive Super Summary* narrative block.
-

-
- Section 2: *Top 3 Cards* (Headline, source publication pill, 3 bullets).
-

-
- Section 3: Call-to-action buttons: *"Read Deep Dive"* and *"Listen to 3-min Audio"*.
-

-
- Footer: 1-click unsubscribe and delivery time settings link.
-

-
- **Batching:** Use asyncio semaphore (max 10 concurrent requests) to comply with Resend rate limits.
-

VERIFICATION COMMAND

```
python3 -m pytest backend/tests/test_email_service.py
```

Expected: HTML renders with valid tags, mock Resend API records successful email dispatch.

Checkpoint 21: Daily Audio Briefing Engine (OpenAI TTS) Multi-Channel

OBJECTIVE & SCOPE

Generate an automated 2–3 minute podcast-style audio briefing every morning using OpenAI TTS, saving to

local storage / Supabase bucket and exposing a streamable endpoint.

FILES TO CREATE / MODIFY

-
- [MODIFY] backend/requirements.txt — Add openai==1.65.0.
-
- [NEW] backend/services/audio_service.py — Audio generation and storage manager.
-
- [MODIFY] backend/main.py — Expose endpoint GET /briefing/today/audio.
-
- [NEW] backend/tests/test_audio_service.py — Test script verifying audio generation.
-

AUDIO PIPELINE SPECIFICATIONS

-
- **Script Generation:** Groq LLM converts the Super Summary + top 3 headlines into a natural radio
-

broadcast script (280–350 words, tone: professional news anchor). Example opening: *"Good*

morning. Here is your NewsBrief intelligence report for [Day]..."

-
- **TTS Synthesis:**
-

```
response = openai_client.audio.speech.create
```

```
model="tts-1"
```

```
voice="onyx", # Deep, executive radio voice
```

```
input=broadcast_script
```

```
response_format="mp3"
```

- **Storage:** Save MP3 to local folder `backend/static/audio/{batch_date}/{summary_id}.mp3` or Supabase

Storage. Populate `super_summaries.audio_url`.

VERIFICATION COMMAND

```
python3 -m pytest backend/tests/test_audio_service.py -v
```

Expected: Generates valid MP3 file >100KB with playable duration between 120s and 180s.

Checkpoint 22: Team Slack Bot & Workspace Integration B2B SaaS

OBJECTIVE & SCOPE

Build the Slack Bot integration for teams (\$99–\$199/mo). Allows teams to connect a Slack channel and

receive automated, beautifully formatted morning Block Kit briefings.

FILES TO CREATE / MODIFY

- [MODIFY] `backend/requirements.txt` — Add `slack-sdk==3.34.0`, `slack-bolt==1.22.0`.
- [NEW] `backend/services/slack_service.py` — Slack Block Kit builder and webhook poster.
- [NEW] `backend/routers/slack_router.py` — OAuth installation routes (`/slack/install`, `/slack/oauth_callback`).

-
- [MODIFY] backend/main.py — Mount slack_router.
-

-
- [NEW] backend/tests/test_slack_service.py — Unit test for Slack Block Kit payload generation.
-

SLACK BLOCK KIT STRUCTURE

```
[{"type": "header", "text": {"type": "plain_text", "text": "🚀 NewsBrief Daily Market Radar" }}, {"type": "section", "text": {"type": "mrkdown", "text": "**Super Summary**\n\n" + summary_text }}, {"type": "divider"}, {"type": "section", "text": {"type": "mrkdown", "text": "**1. " + card1.headline + " " + card1.subheadline + "\n\n" + card1.bullet1 }}, {"type": "actions", "elements": [{"type": "button", "text": {"type": "plain_text", "text": "Read Deep Dive" }, "url": web_url, "style": "primary"}]}]
```



VERIFICATION COMMAND

```
python3 -m pytest backend/tests/test_slack_service.py -v
```

Expected: Block Kit generator passes Slack schema validation and mock WebClient executes chat.postMessage.

Checkpoint 23: Frontend Modernization & Audio Player Mobile / Expo

OBJECTIVE & SCOPE

Update the React Native / Expo application to include an audio playback widget on the home screen, an

email digest preference switch, and an overhauled 4-tier paywall with a 1-click "Expense to Company"

receipt generator.

FILES TO CREATE / MODIFY

- [MODIFY] frontend/package.json — Add expo-av@~15.0.2.
- [NEW] frontend/src/components/AudioPlayer.tsx — Play/Pause audio bar with progress indicator.

-
- [MODIFY] frontend/src/screens/HomeScreen.tsx — Embed AudioPlayer component above the Super
-

Summary card.

-
- [MODIFY] frontend/src/screens/PaywallScreen.tsx — Replace single \$4.99 card with 4-tier selection: Free
-

(\$0), Pro (\$9.99), Executive (\$24.99), Team (\$99). Add "Expense to Employer" modal.

-
- [NEW] frontend/src/components/ExpenseModal.tsx — Modal rendering formal PDF/text invoice for
-

employer expensing.

-
- [MODIFY] frontend/src/types.ts — Add audio_url to BriefingResponse and subscription tier enums.
-

PAYWALL PRICING LADDER & UI LOGIC

```
export const TIERS = [
```

```
{ id: 'pro', name: 'Pro', price: '$9.99/mo', desc: 'Unlimited Cards + 2 Deep Dives/day + Daily Audio' }
```

```
{ id: 'executive', name: 'Executive', price: '$24.99/mo', desc: 'Unlimited Deep Dives + Read as One + Priority arXiv' }
```

```
{ id: 'team', name: 'Team / Slack', price: '$99/mo', desc: 'Up to 15 seats + Daily Automated Slack Bot Drop' }
```

VERIFICATION STEPS

-
- Run `npm test` or launch Expo: verify `AudioPlayer` mounts without crashes when `audio_url` is present.
-

-
- Click "Expense to Employer" in `PaywallScreen`: verify formatted reimbursement receipt displays
-

correctly.

Checkpoint 24: Distributed Task Queue (Redis + Arq) Infrastructure

OBJECTIVE & SCOPE

Replace in-process `APScheduler` with a production-grade Redis + arq distributed task queue. Manages

nightly Stage 1 runs and fans out Stage 2 deliveries by user timezone without blocking the API.

FILES TO CREATE / MODIFY

-
- [MODIFY] `backend/requirements.txt` — Add `arq==0.26.1`, `redis==5.2.1`.
-
- [NEW] `backend/worker.py` — Arq worker definitions, task handlers, and cron schedules.
-
- [MODIFY] `backend/main.py` — Remove in-memory `APScheduler`; connect to Redis connection pool.
-
- [NEW] `docker-compose.yml` — Docker compose file orchestrating FastAPI app, Redis server, and Arq
-

worker.

-
- [NEW] `backend/tests/test_worker_tasks.py` — Task execution tests.
-

WORKER TASK HIERARCHY (BACKEND/WORKER.PY)

```
class WorkerSettings:
    """Worker Settings"""

    functions = {
        'run_stage1_pipeline_task': run_stage1_pipeline_task,
        'dispatch_user_delivery_task': dispatch_user_delivery_task,
        'send_email_digest_task': send_email_digest_task,
        'post_slack_briefing_task': post_slack_briefing_task,
    }

    cron_jobs = {
        # Run Stage 1 globally at 03:00 UTC
        'run_stage1_pipeline_task': cron(run_stage1_pipeline_task, hour=3, minute=0),

        # Trigger Stage 2 deliveries hourly based on timezone
        'dispatch_hourly_deliveries_task': cron(dispatch_hourly_deliveries_task, minute=0),
    }

    redis_settings = RedisSettings.from_dsn(os.environ.get("REDIS_URL", "redis://localhost:6379"))
```

VERIFICATION COMMAND

```
python3 -m pytest backend/tests/test_worker_tasks.py
```

Expected: Tasks enqueue to Redis, execute, and record completion status.

Checkpoint 25: Admin Dashboard 2.0 & Telemetry Admin & Ops

OBJECTIVE & SCOPE

Overhaul admin.html into a full operational command center displaying source health badges, daily LLM

token costs, multi-channel delivery funnels, and manual trigger controls.

FILES TO CREATE / MODIFY

- [MODIFY] backend/admin.html — Revamped single-page UI with Tailwind CSS CDN and live telemetry

charts.

- [MODIFY] backend/main.py — Expand GET /admin/stats to return source ingestion metrics, API cost

estimates, and multi-channel counts.

- [NEW] backend/tests/test_admin_endpoints.py — Verification of admin stats and pipeline triggers.

ADMIN TELEMETRY DATA CONTRACT (GET /ADMIN/STATS)

```
"sources_health": {
```

"total_raw_articles": 110

```
"clusters_formed": 48,
```

"emails_dispatched": 340,

```
"estimated_cost_today_usd": 0.42, # Groq + TTS + Resend
"active_subscriptions": { "free": 450, "pro": 62, "executive": 18, "team": 8 }
```

VERIFICATION COMMAND

```
python3 -m pytest backend/tests/test_admin_endpoints.py -v
```

Expected: GET /admin/stats returns full telemetry object with 200 OK.

3. Complete Master Verification Checklist

CP	Module	Primary Deliverable	Verification Command
CP-16	Ingestion	HN, GitHub, arXiv, RSS scrapers in backend/ingestion_sources/	pytest backend/tests/test_ingestion_sources.py
CP-17	Database	PostgreSQL pgvector, Team, SlackInstallation tables	pytest backend/tests/test_database_vector.py
CP-18	Pipeline 1	Stage 1 global pool clustering & pre-generation in pipeline_stage1.py	pytest backend/tests/test_pipeline_stage1.py
CP-19	Pipeline 2	Stage 2 pgvector cosine matching in pipeline_stage2.py (<1.5s/user)	pytest backend/tests/test_pipeline_stage2.py
CP-20	Email	Resend API integration with responsive HTML digest template	pytest backend/tests/test_email_service.py
CP-21	Audio	OpenAI TTS 2–3 min daily radio podcast generator	pytest backend/tests/test_audio_service.py
CP-	Slack	Slack Bolt OAuth & Block Kit morning briefing in #market-	pytest

CP	Module	Primary Deliverable	Verification Command
22		intel	backend/tests/test_slack_service.py
CP-23	Mobile UI	AudioPlayer.tsx on Home + 4-Tier Paywall & Employer Receipt Modal	npm test (frontend)
CP-24	Queue	Redis + <code>arq</code> worker pool & timezone-aware scheduler	pytest backend/tests/test_worker_tasks.py
CP-25	Admin	Admin Dashboard 2.0 with source health and API token spend tracker	pytest backend/tests/test_admin_endpoints.py